ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – ГАБРОВО
ФАКУЛТЕТ ПО ЕЛЕКТРОТЕХНИКА И ЕЛЕКТРОНИКА
Специалност „Софтуерно и компютърно инженерство“

Дисциплина: Нерелационни Бази Данни
летен семестър, 2025/2026

Тема: *Проектиране на мащабируема база данни за социална мрежа*

КУРСОВ ПРОЕКТ

Автор: *Богомил Георгиев Иванов*, фак. номер: *22372126*

Април 2026

Габрово

Съдържание

1. Въведение

В съвременния дигитален свят социалните мрежи обработват огромни масиви от динамични данни в реално време – публикации, коментари и сложни мрежи от приятелства. Традиционните релационни бази данни често срещат сериозни затруднения с бързодействието и хоризонталното мащабиране при управлението на такъв тип неструктурирана информация.

Този проект решава проблема чрез проектиране на мащабируема база данни за социална мрежа. За целта е използвана документно-ориентирана база данни MongoDB. Тя осигурява гъвкава схема, позволяваща работа с вложени документи, и предлага отлични възможности за мащабируемост. Цялостното решение е реализирано като уеб-базирано приложение, използвайки MERN стек, който включва: MongoDB, Express.js, React и Node.js.

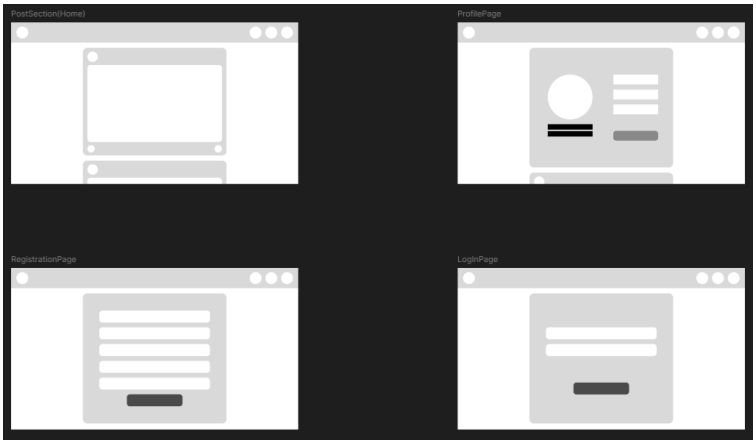
2. Анализ на решението

2.1. Потребителски изисквания и работен процес

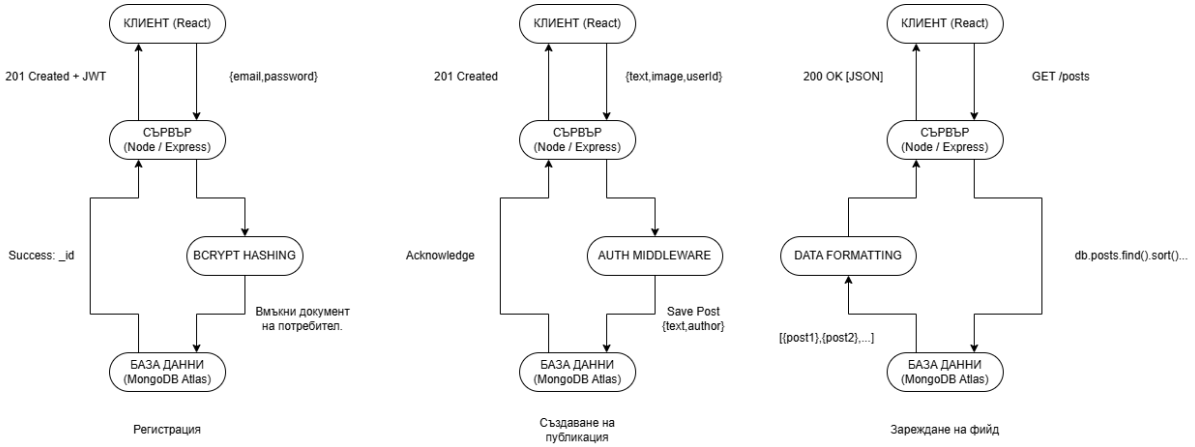
Тук в тази секция ще опиша най-общото представяне на работния процес, свързан с базата данни. Ще го разгледаме на отделни секции: *входно съдържание*, *обработка и запазване в системата* и *изходното съдържание*.

- *Входно съдържание*: Данните се въвеждат от потребителя чрез форми в графичния интерфейс (React приложението). Основният вход включва регистрационни данни на потребителите (име, имейл, парола), текстово съдържание за нови публикации и текстово съдържание за коментари към съществуващи публикации. Тези данни се получават директно от действията на потребителя в браузъра.
- *Обработка и запазване в системата*: Входните данни се изпращат като JSON обекти чрез HTTP заявки към сървърната част (използвайки Node.js/Express.js). Там те се валидират и чрез обектно-релационен *mapper* (като Mongoose) се записват в съответните колекции на MongoDB (Users, Posts, Comments). Избираме стратегия с *реферирание* (referencing) между колекциите, за да се избегне претоварване на документите при голям брой коментари. (Повече обяснено в *2.4.*)
- *Изходно съдържание:* Изходът ще е *агрегирана информация* – потребителски фийд с публикации и прилежащите им коментари. Тази информация се извлича от базата данни, подава се обратно към React клиента и се визуализира в графичния интерфейс, за да бъде четена и взаимодействана от потребителите.

2.2. Примерен графичен интерфейс



2.3. Диаграми на анализа



2.4. Модел на съдържанието / данните

За да изградим базата данни, която работи с frontend-а, трябва да създадем модел, по който ще се ориентираме какви данни се изискват от потребителя и какви данни ще са необходими, за да може потребителя да достъпи профила си в нашата социална мрежа.

Към нашия модел трябва да разгледаме какви стратегии за проектиране ще са най-ефективни. Има два вида: embedding и referencing. Този избор е много важен и за това ще разгледаме какво представляват и ще разгледаме предимствата им и недостатъците им.

При влагането (embedding) всички свързани данни се запазват в един общ документ. Този подход е напълно подходящ, ако искаме бързодействие за нашата база данни, но в нашия случай ние използваме MongoDB. MongoDB и други NoSQL бази данни от документен тип имат лимит за размер на един документ. Ако документа се препълни, няма да можем да записваме нова информация.

Вместо това имаме и реферирание (referencing). Чрез този подход данните се разделят в различни колекции, като се свързват помежду си чрез уникален идентификатор. Основното предимство на този подход е безкрайната мащабируемост (точно това, което търсим). Също документите остават малки, леки и няма да стигнат лимита. Недостатъкът обаче е, че изисква допълнителна работа за сървъра. За да покажем името на автора на публикацията или коментарите, ще се наложи да използваме допълнителни заявки като .populate() (JOIN в SQL). Тези заявки изразходват повече изчислителна мощ, но за сметка на това имаме мащабируема база данни.

Ще използваме реферирание и ще проектираме нашата база данни да съдържа три колекции:

- ✓ *Потребители (Users)*: Отговаря за съхранението на идентификационните данни, включително уникално потребителско име, имейл и криптирана парола;
- ✓ *Публикации (Posts)*: Съдържа текстовото съдържание на конкретния пост, времеви печат за създаването му и референция (ObjectId) към потребителя, който го е създал;

✓ *Коментари (Comments)*: Съхранява текста на коментара и двойна референция - една към публикацията (към която принадлежи) и една към автора му.

3. Дизайн

Програмната реализация на сървърната част е изградена чрез Node.js и Express.js, като за връзка с базата данни използваме обектно-документираният мапер (ODM) Mongoose. За управлението на данните са проектиране REST API маршрути(routes), които обслужват основните CRUD операции.

За да решим проблема с разделените данни (поради стратегията реферирane), при извличането на публикациите и коментарите се използва функционалността за агрегиране. Чрез метода .populate() сървърът динамично обединява данните, извличайки потребителското име на автора чрез неговия уникален идентификатор. То е малко или много подобно на JOIN операциите в релационните бази данни, обаче за нерелационна среда.

```
app.get('/api/posts', async (req, res) => {
  try {
    const posts = await Post.find()
      .populate('author', 'username')
      .sort({ createdAt: -1 });
    res.status(200).json(posts);
  } catch (error) {
    res.status(500).json({ error: 'Грешка при извличане' });
  }
});
```

За да се гарантира бързото действие на системата при нарастващ обем от данни, на ниво схема сме имплементирали специфични индекси. Например, върху датата на създаване на публикациите е приложен низходящ индекс, което драстично ускорява генерирането на хронологичен потребителски фийд.

```
const mongoose = require('mongoose');
const PostSchema = new mongoose.Schema({
  content: { type: String, required: true },
  author: { type: mongoose.Schema.Types.ObjectId, ref: 'User',
required: true }
}, { timestamps: true });
PostSchema.index({ createdAt: -1 });
module.exports = mongoose.model('Post', PostSchema);
```

4. Тестване

За да се гарантира надеждността и бързодействието на системата, са предвидени следните етапи и сценарии за тестване:

- *Функционално тестване на графичния интерфейс*: Проверка на формите за регистрация, създаване на публикации и добавяне на коментари. Тестването ще се проведе в различни съвременни уеб браузъри (Google Chrome, Mozilla Firefox, Safari), за да се гарантира съвместимостта на React компонентите.
- *Тестване на сървърната логика (API)*: Проверка на всички основни CRUD операции (създаване, четене, обновяване, изтриване) чрез симулиране на HTTP заявки и валидация на върнатите статус кодове.
- Анализ на производителността и оптимизация чрез explain(): Това е ключов етап от тестването на мащабируемостта на избраната нерелационна база данни. За да се докаже ефективността на дефинираните индекси (например върху createdAt за сортиране на фийда и username за търсене), към заявките се прилага методът explain("executionStats").

Резултатите от него доказват, че при използване на индекс, заявката преминава през стадий IXSCAN (Index Scan) и сканира само нужния брой документи, вместо да обхожда цялата колекция (COLLSCAN). Това драстично намалява времето (в милисекунди) за изпълнение на сложните агрегационни pipeline-ове при генериране на потребителски фийд и потвърждава, че архитектурата е готова да поеме голям обем от данни ефективно.

5. Заключение и възможно бъдещо развитие

В настоящия курсов проект беше успешно проектирана и реализирана мащабируема нерелационна база данни за социална мрежа. Използването на MongoDB в комбинация с MERN стек (Express.js, React, Node.js) се доказа като изключително ефективно решение за управление на динамично съдържание.

Предимства и ограничения на използваните технологии:

Основното предимство на реализираната архитектура е нейната гъвкавост и защита от претоварване на документите чрез използването на реферирани колекции. Системата успешно обслужва създаване на публикации, динамично агрегиране на данни за авторите, интерактивни коментари и визуално отразяване на харесвания в реално време на клиентската страна. Като ограничение може да се отчете необходимостта от допълнителни агрегационни заявки на ниво сървър за обединяване на данните, което е по-ресурсоемко в сравнение с директното им влагане.

Насоки за бъдещо развитие:

Проектираната система има отлична база за надграждане. Като насоки за бъдещо развитие на решението се предвижда въвеждане на гео-локация за търсене и филтриране на публикации по специфичен регион, както и интеграция на WebSockets протокол, с който да се „оживи“ вече подготвената графична секция за изпращане на лични съобщения между потребителите в реално време.

6. Използвани литературни източници и Уеб сайтове

[1] Уеб сайт на MongoDB (Официална документация относно Data Modeling), адрес: https://www.mongodb.com/docs/manual/core/data-modeling-introduction/

[2] Уеб сайт на React JS (Официална документация), адрес: https://react.dev/

[3] Chodorow, K. (2013). MongoDB: The Definitive Guide. O'Reilly Media.

[4] Sadalage, P. J., & Fowler, M. (2012). NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence. Addison-Wesley Professional.